

Arduino语音信号采集

电子信息工程学院
空天电子信息国家级实验教学示范中心
空天电子信息国家级虚拟仿真实验教学中心





实验内容

- 音频放大电路实验
 - 测量传声器输出
 - 测试单级放大电路输出
 - 控制信号的直流分量
- Arduino采集语音信号



实验目的

- 学习声音传感器原理
 - 驻极体传声器的结构与参数
 - 声电转换过程
- 理解运算放大器原理
 - 搭建LM358芯片的放大电路
 - 计算放大电路的增益
- 使用单片机接收语音模拟信号
 - Arduino模数转换



基本名词解释

■ 驻极体 (Electret)

- 能长久保持电极化状态的电介质。这种电介质一般是高分子聚合物。例如：聚丙烯、聚四氟乙烯等

■ 传声器 (Microphone)

- 将声音信号转换为电信号的换能器

- 按工作原理分类
 - 动圈式
 - 电磁式
 - 压电式
 - 电容式
 - 声频电容传声器
 - 驻极体电容传声器

■ 驻极体电容传声器 (ECM)

- 俗称咪头，是一种声电换能器





基本名词解释-ECM 分类

驻极体电容传声器
(ECM) 分类:

振膜式 (Foil)

背极式 (Back)

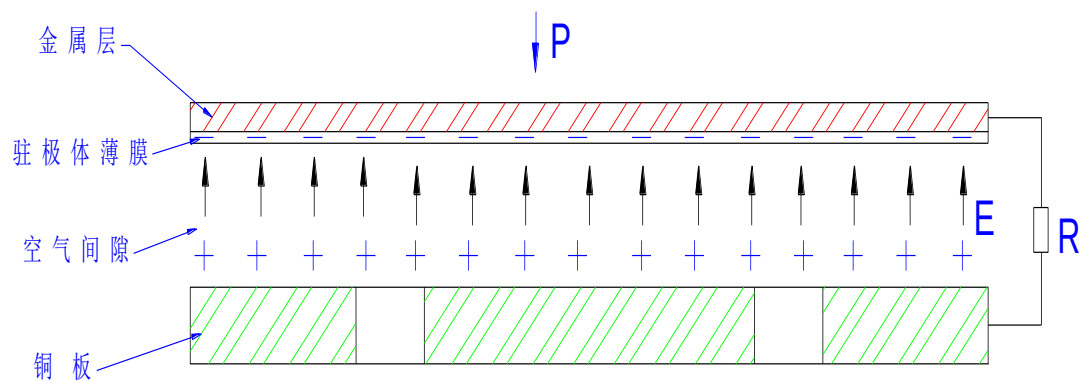
前极式 (Front)



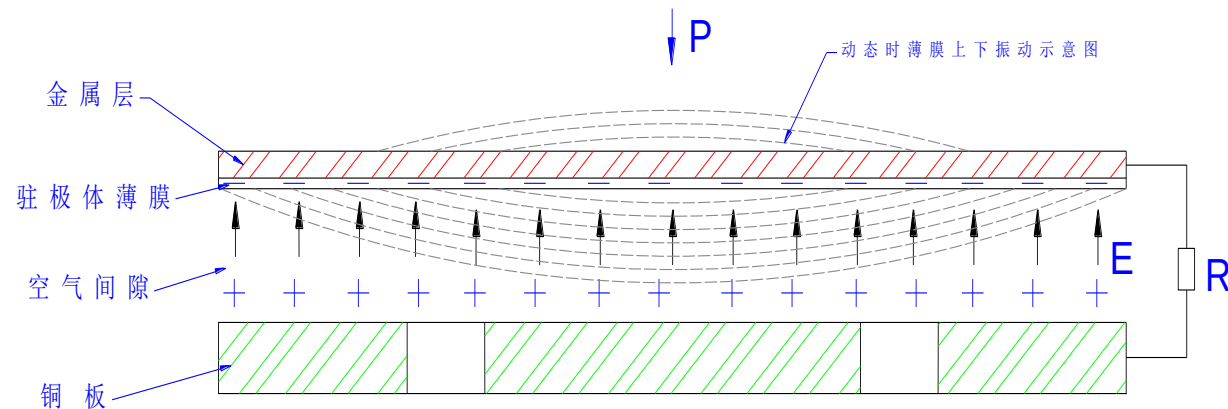
驻极体电容传声器工作原理

- 镀金属层薄膜与背极板形成空气介质电容
- 对驻极体充电形成电场： $U = \frac{Q}{C}$
- 声波使薄膜振动，改变电容和电场，产生电信号 $\Delta U = \frac{Q}{\Delta C}$

静态：形成电场

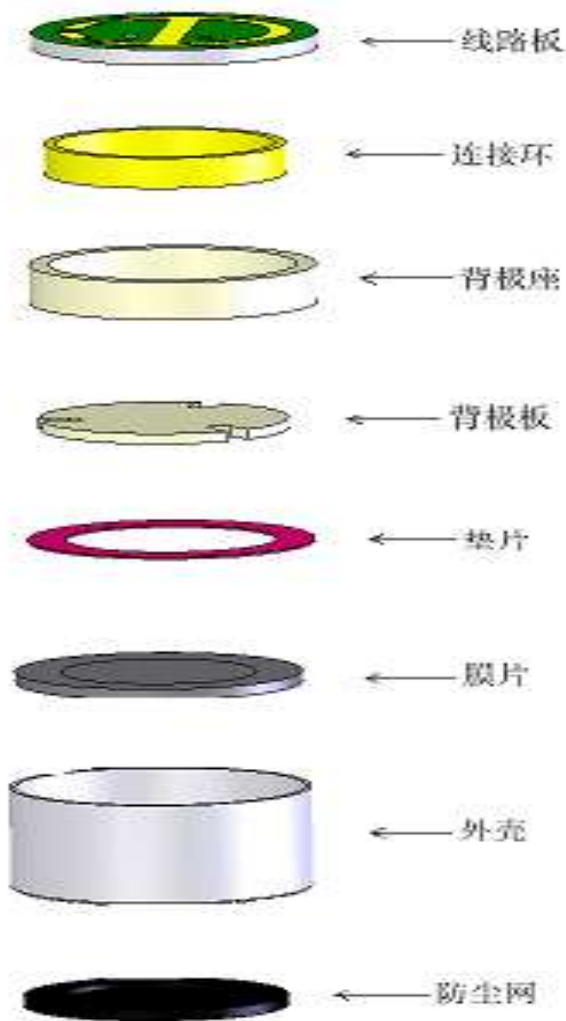


动态：电场随声波强度改变





驻极体电容传声器结构示意图





驻极体传声器重要参数

■ 灵敏度 (Sensitivity)

- 表示传声器的声——电转换效率
- 在自由声场中，当向传声器施加一个声压为1帕 (Pa) 或 1微巴 (ubar) 的声信号时，传声器的开路输出（以毫伏为单位），即为该传声器的灵敏度。
- 注：1微巴 (ubar) 约相当于人们正常音量讲话。



驻极体传声器重要参数

- 1微巴 (ubar) 约相当于人们正常音量讲话
 - 并在离嘴1米远的地方测量所得到的声压
- 灵敏度的单位
 - 毫伏/帕 (mv / Pa) —— 国际标准
 - 毫伏/微巴 (mv / ubar) —— 日本标准
- 1帕 (Pa) 是指1牛顿 (N) 的力作用在1平方米面积上的压强
- Pa与ubar的换算关系为
 - $1\text{Pa} = 10\text{ubar}$
 - $1\text{mv}/\text{ubar} = 10\text{mv}/\text{pa}$



驻极体传声器重要参数

■ ECM灵敏度参数一般用灵敏度级表示

■ 单位为分贝 (dB)

■ 公式：
$$Lm = 20 \log_{10} \left(\frac{M}{Mr} \right)$$

➤ M : 灵敏度 Mr : 参考灵敏度 (0dB = 1V / Pa)

■ 例：

$$M = 10mV / Pa$$

$$Lm = 20 \log_{10} \left(\frac{10mV / Pa}{1V / Pa} \right)$$

$$= 20 \times (-2)$$

$$= -40dB$$



驻极体传声器灵敏度

■ 准确测量灵敏度的条件：

➤ ① 测量环境

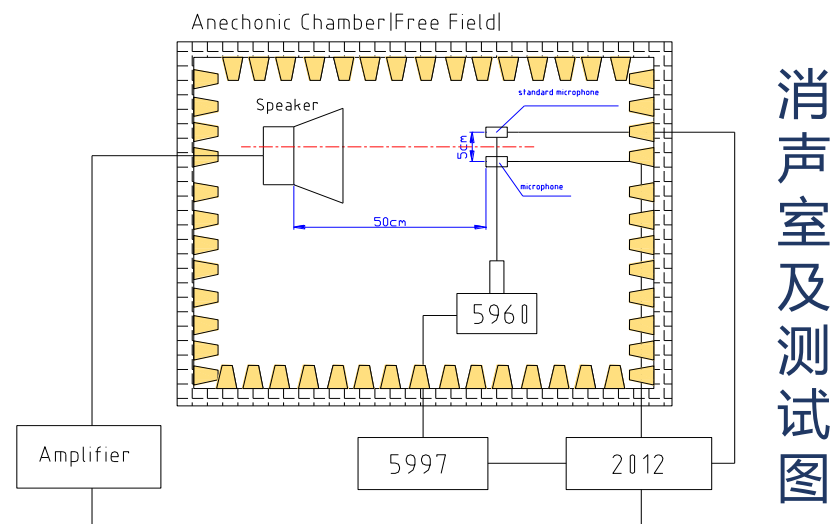
- 正常测试条件

- 温度：5~35°C
- 湿度：45~85%
- 气压：86~106kPa

- 标准测试条件

- 温度：20±2°C
- 湿度：60~70%
- 气压：86~106kPa

② 声学环境要求：自由场——消声室



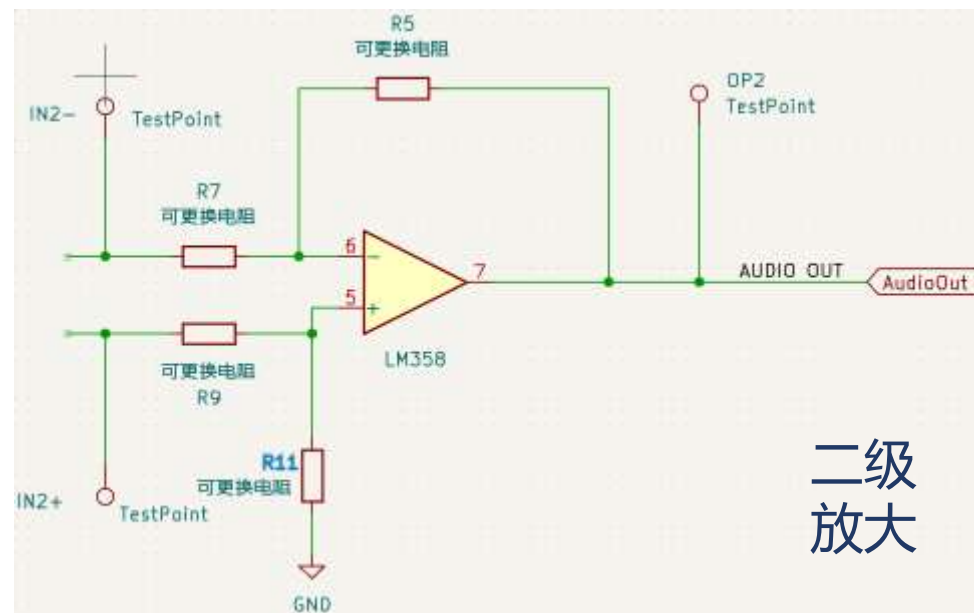
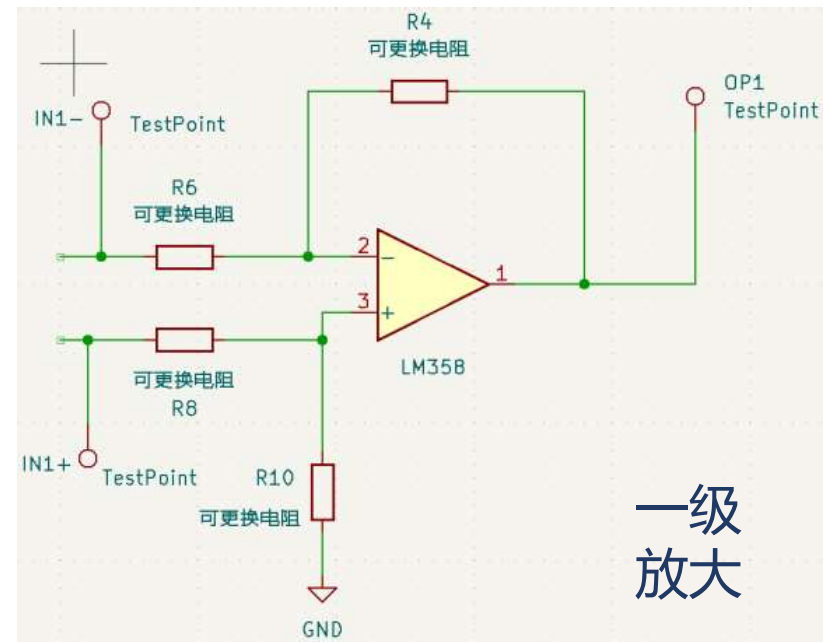
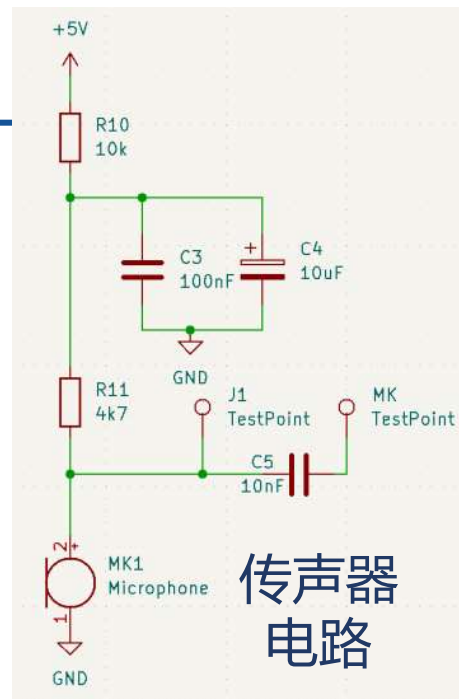
③ 测试标准仪器：音频信号分析仪





实验电路板说明

■ 麦克风及运放板

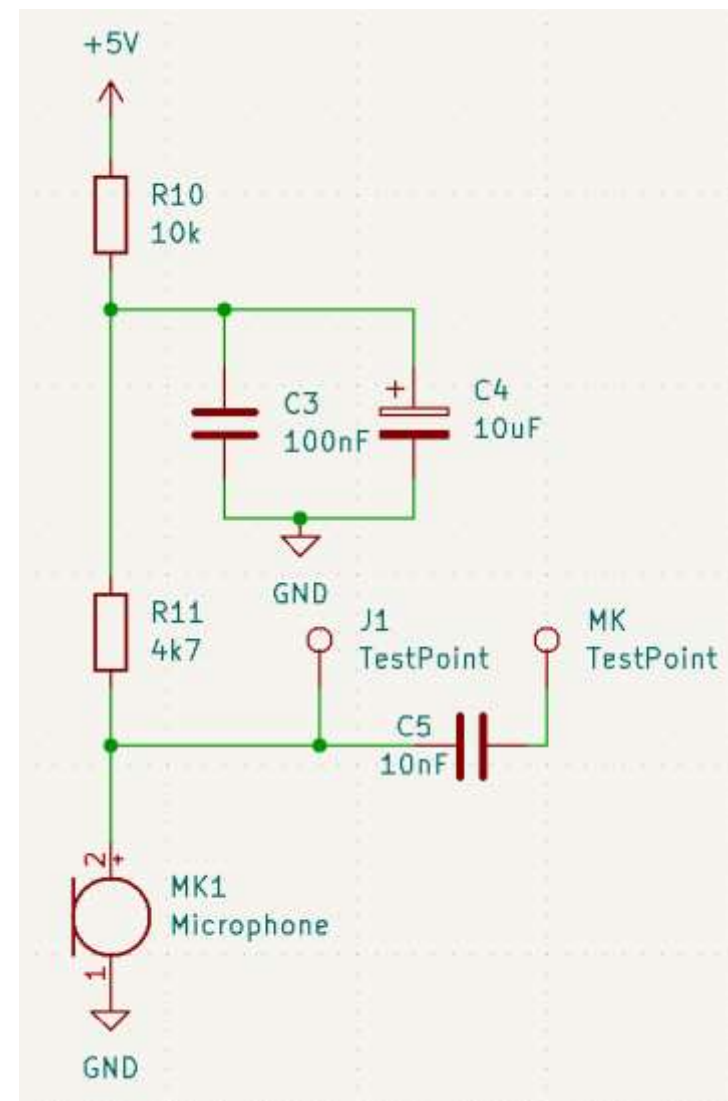
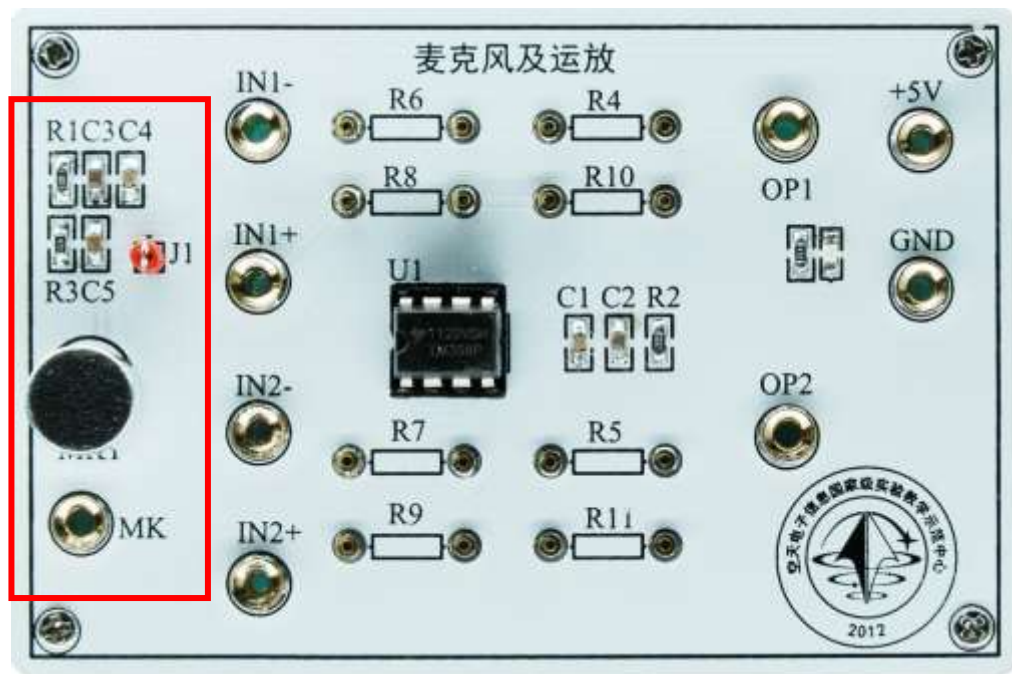


二级放大



实验一：测量传声器输出

■ 音频输入-传声器电路

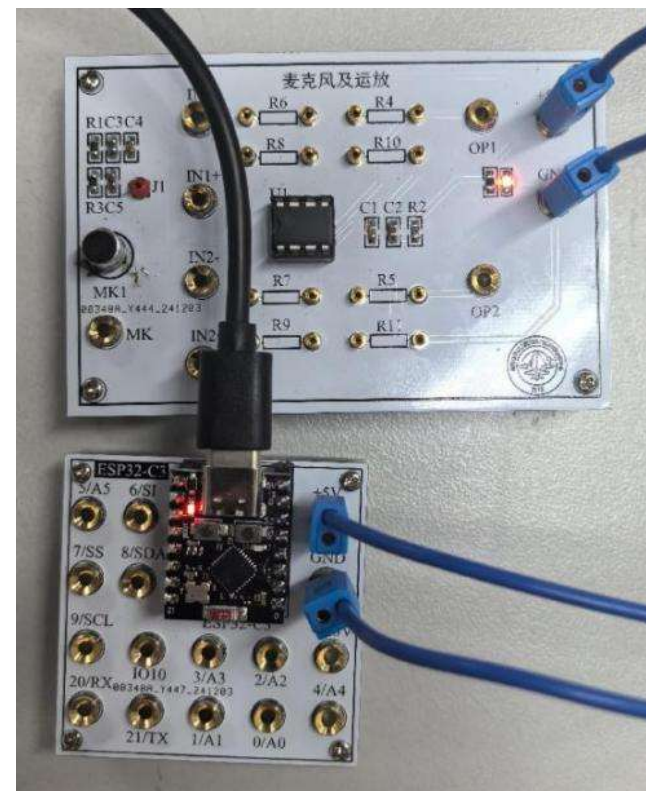
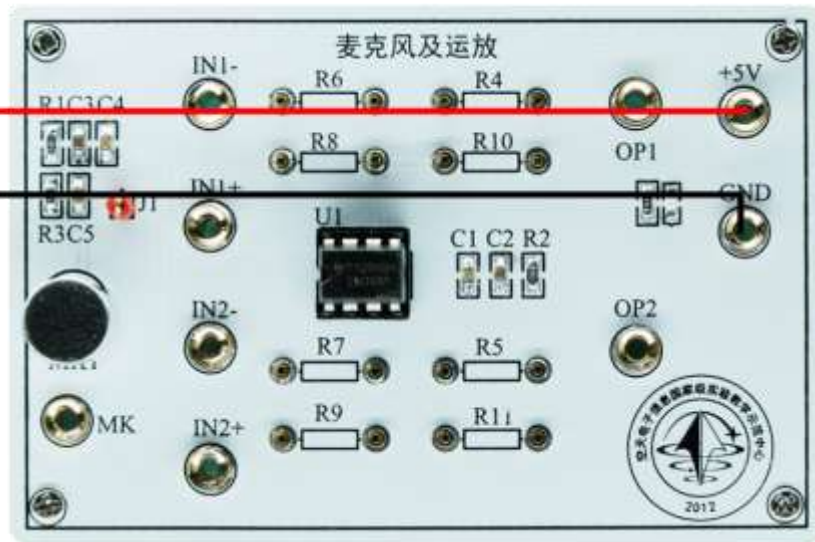




实验一：测量传声器输出

■ 步骤1：给麦克风及运放板供电

- 利用ESP32-C3核心板进行间接供电
- ESP32-C3核心板**+5V**接麦克风及运放板**+5V**
- ESP32-C3核心板**GND**接麦克风及运放板**GND**





实验一：测量传声器输出

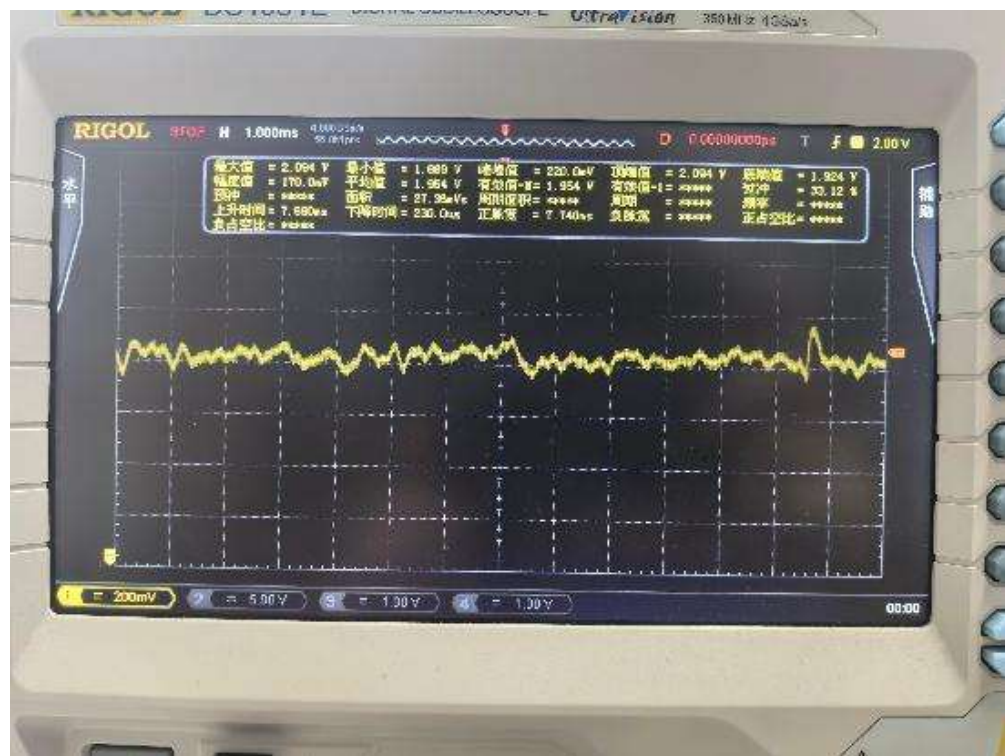
■ 步骤2：用示波器分别测量J1和MK端口的输出

- J1端口可以通过示波器红色夹子直接连接
- MK端口通过2mm香蕉头引出再连接示波器
- 选择喊出以下声音：
 - 开灯、关灯、你好、再见





实验一：测量传声器输出



J1端子测量结果参考

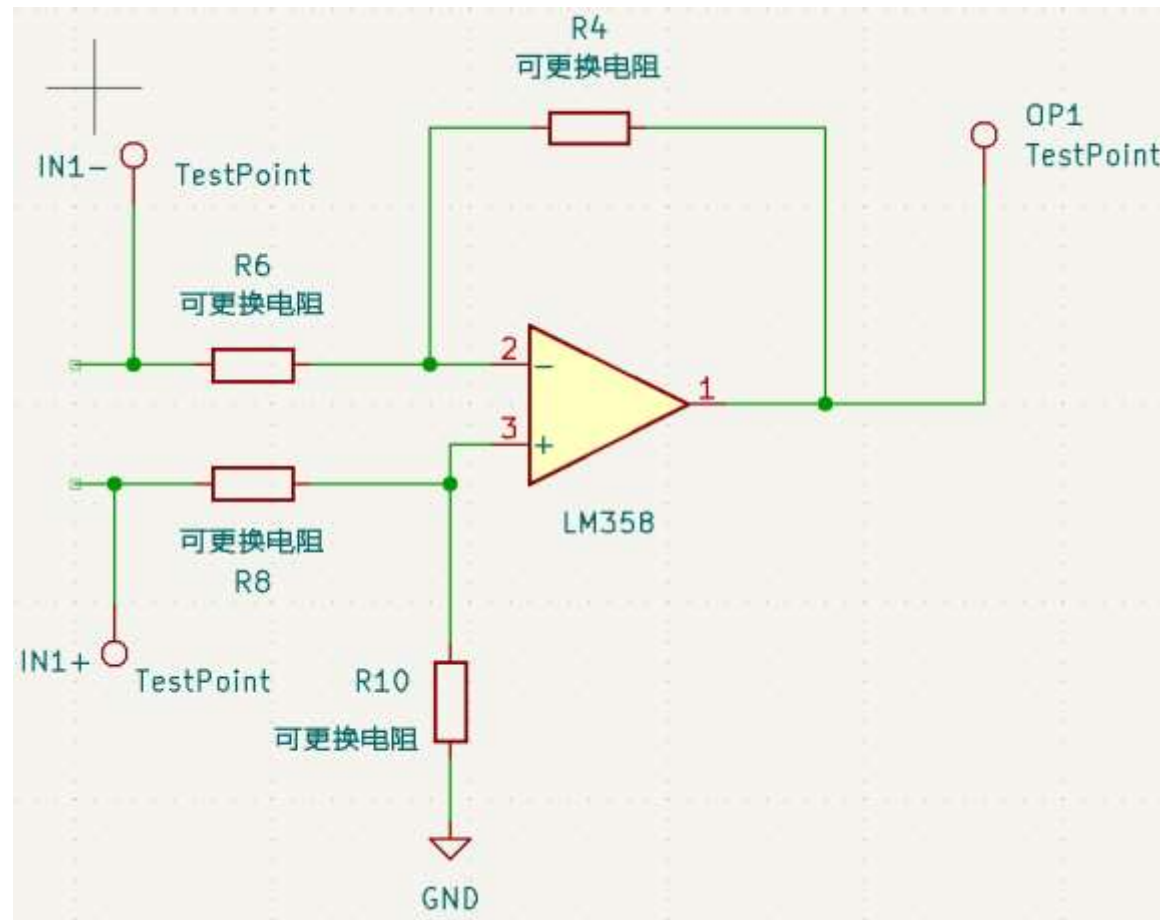
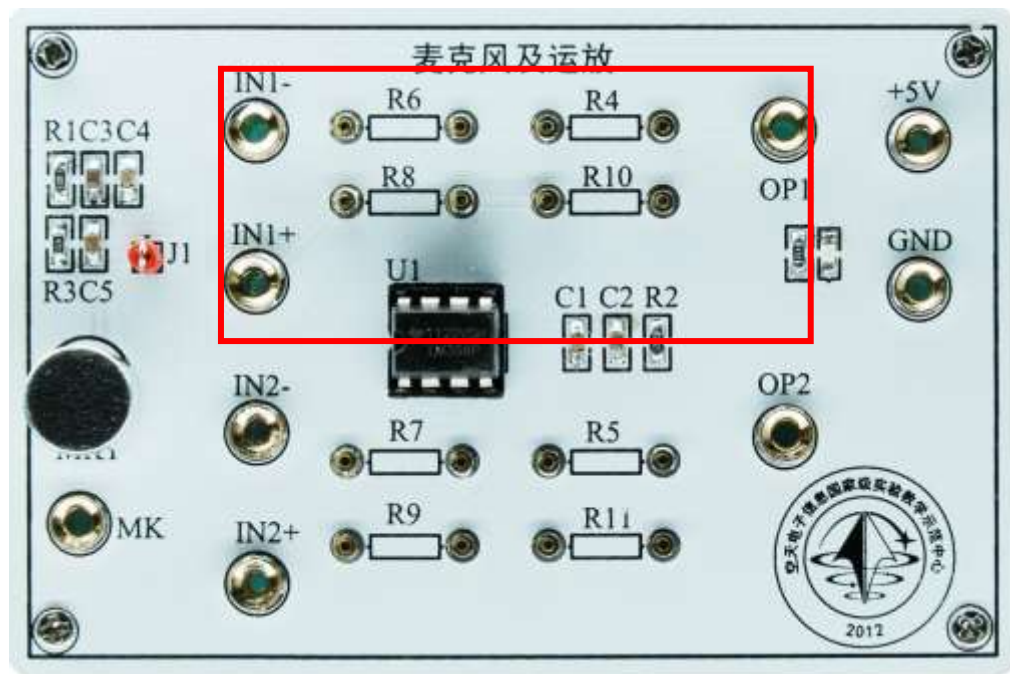


MK1端口测量结果参考



实验二：测试单级放大电路输出

■ 放大电路——一级放大





实验二：测试单级放大电路输出

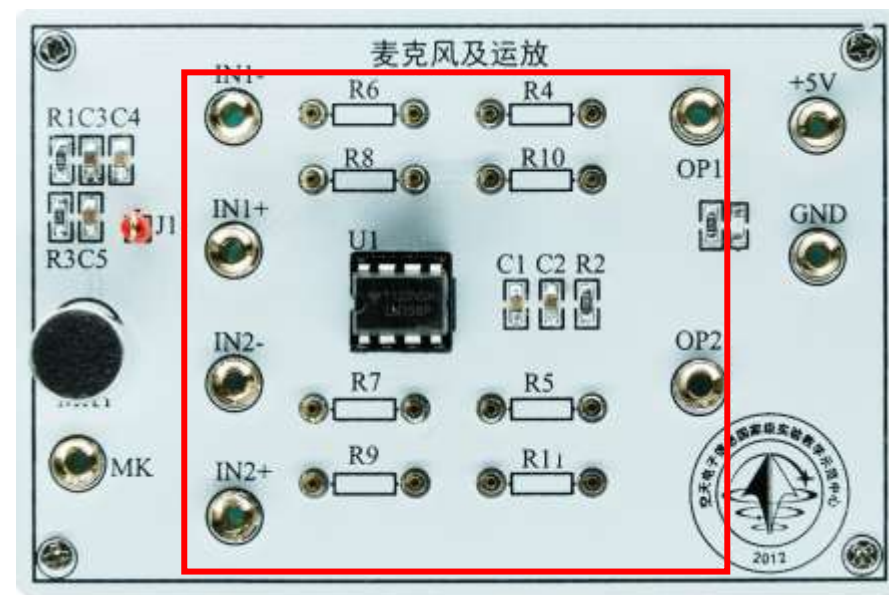
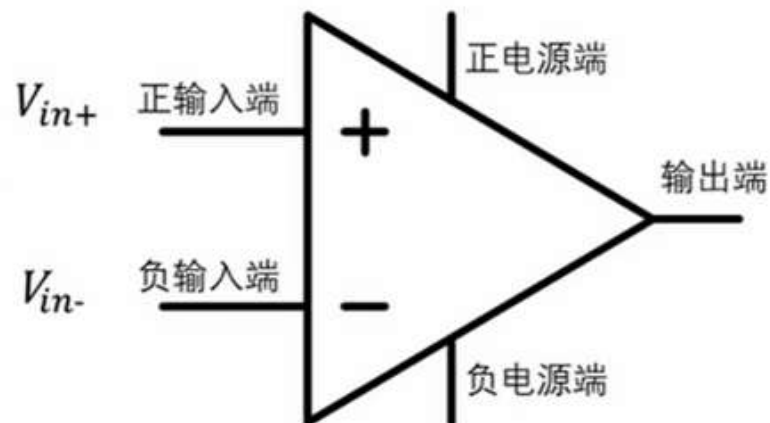
■ 运算放大器原理

➤ 五个端口

- 同相输入端 (V_{in+}) (正输入端)
- 反相输入端 (V_{in-}) (负输入端)
- 正电源端 (V_{CC})
- 负电源端 (V_{EE})
- 输出端 (V_{out})

➤ 理想运算放大器模型

- $V_{out} = A(V_{in+} - V_{in-})$
- 其中，增益（放大倍数） $A = \infty$





实验二：测试单级放大电路输出

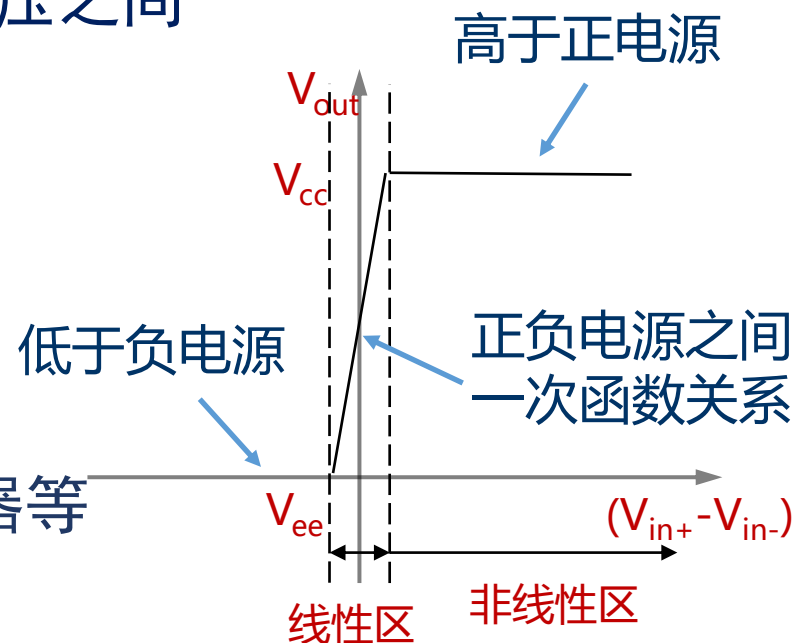
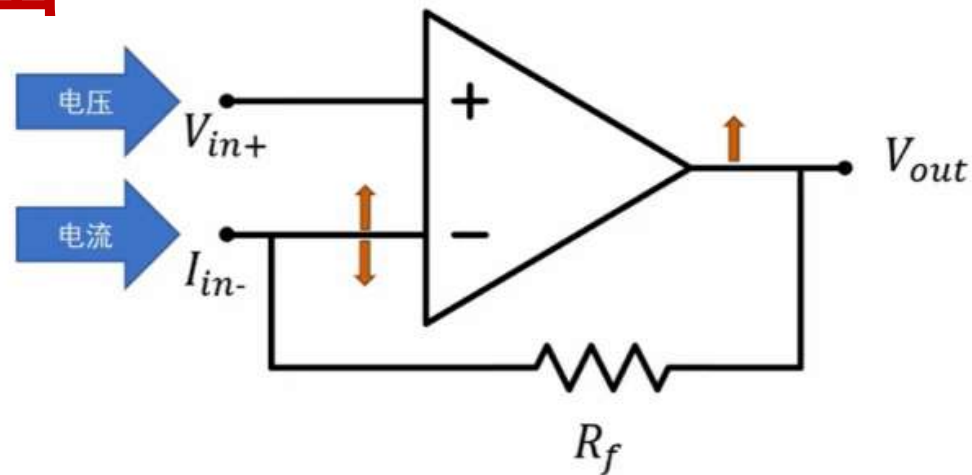
■ 运算放大器原理

➤ 实际的放大器模型

- $V_{out} = A(V_{in+} - V_{in-})$
- 无限放大倍数 $A = \infty$ 不再成立
- $A = 50000 \sim 100000$ ，且随温度、个体差异变化极大
- 运算放大器的输出端电压必须落在正负电源电压之间
 - 电源电压决定输出端电压的上下限

➤ 放大器的稳定方法：引入负反馈

- 负反馈让运放的放大倍数稳定在某个值
 - 这个值由外部电路确定
- 改变外部电路可以让运放实现不同功能
 - 同相放大器、反相放大器、积分器、微分器等





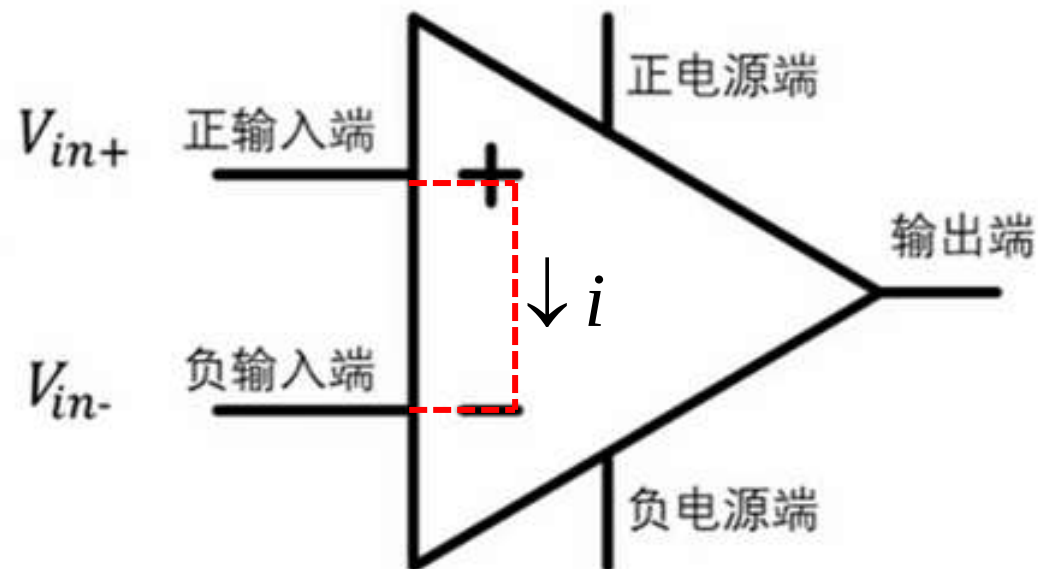
实验二：测试单级放大电路输出

■ 运算放大器原理

➤ 虚短与虚断

➤ 当运算放大器处于负反馈状态下的两个假设

- **虚短**： $V_{in+} = V_{in-}$ （运放的正输入端和负输入端**电压相等**）
- **虚断**： $i = 0$ （通过正负输入端流入运放的**电流为0**）





实验二：测试单级放大电路输出

■ 基本放大电路

➤ 同相放大器

- 根据欧姆定律

$$i_1 = \frac{V_{in-}}{R_6}, i_2 = \frac{V_{out} - V_{in-}}{R_4}$$

- 根据“虚断”， V_{in+} 不向运放内输入电流

$$V_{in+} = V_{in}$$

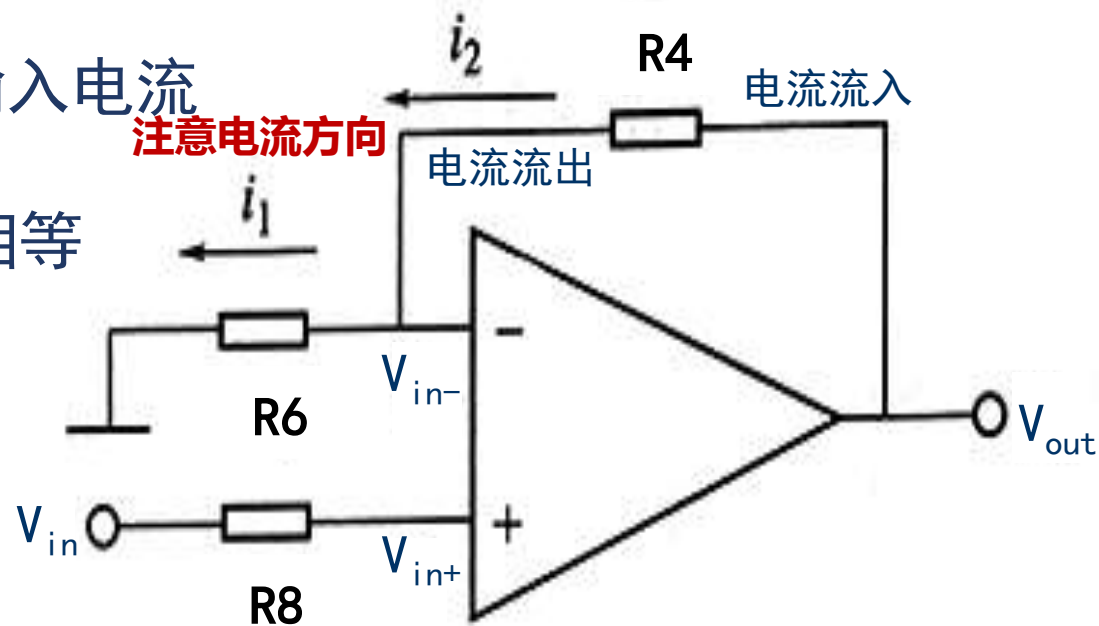
- 根据“虚短”，运放两输入电压相等

$$V_{in+} = V_{in-}$$

- 联立以上式子，得到

$$V_{out} = \frac{R_4 + R_6}{R_6} V_{in}$$

$$i = \frac{\Delta U}{R}, \Delta U \text{ 为电流流入点的电压减电流流出点的电压}$$





实验二：测试单级放大电路输出

■ 基本放大电路

➤ 反相放大器

- 根据欧姆定律

$$i_1 = \frac{V_i - V_{in-}}{R_6}, i_2 = \frac{V_{in-} - V_{out}}{R_4}$$

- 根据“虚断”， V_{in+} 不向运放内输入电流

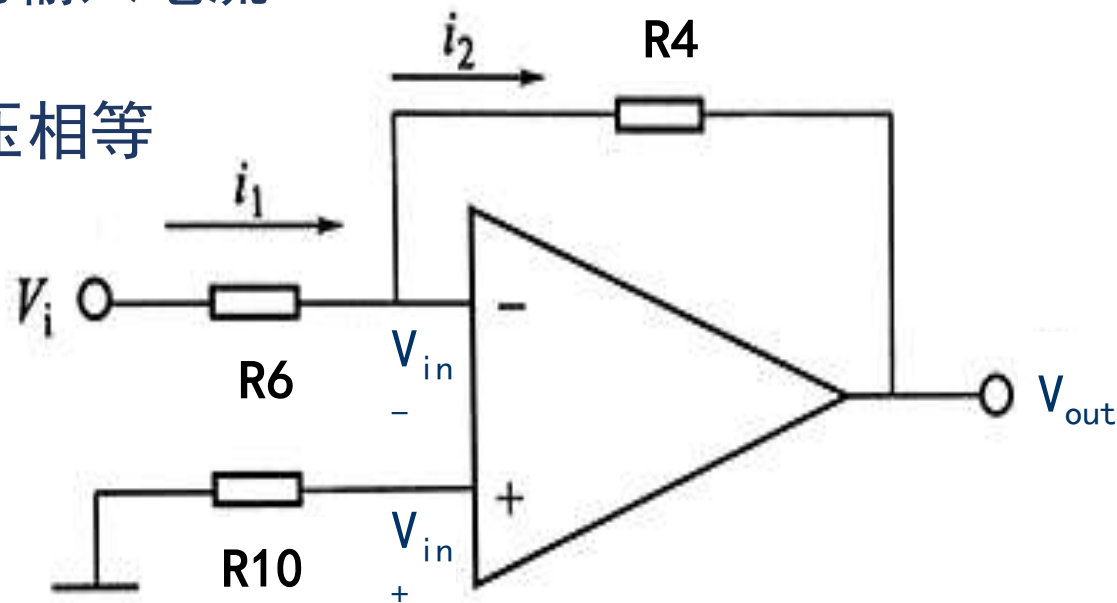
$$V_{in+} = V_{in}$$

- 根据“虚短”，运放两输入电压相等

$$V_{in+} = V_{in-}$$

- 联立以上式子，得到

$$V_{out} = -\frac{R_4}{R_6} V_{in}$$





实验二：测试单级放大电路输出

■ 实验使用的运算放大器

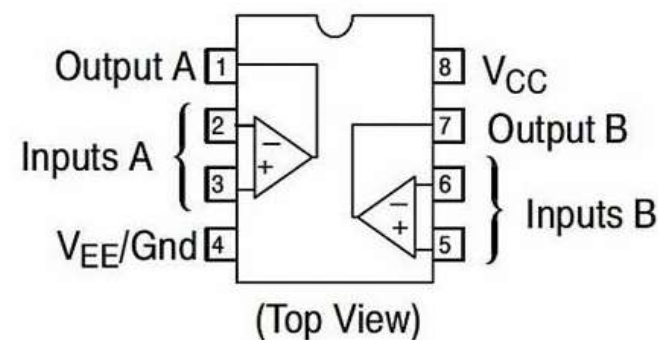
➤ LM358运算放大芯片

- LM358是一种低功率双运算放大芯片
- 包含**两颗运算放大器**
 - 第一颗：同相输入端（3），反相输入端（2），正电源端（8），负电源端（4），输出端（1）
 - 第二颗：同相输入端（5），反相输入端（6），正电源端（8），负电源端（4），输出端（7）
- 两个运算放大器共用8、4引脚供电
- 本实验中**正电源端接5V电压，负电源端接地**

电源电压决定输出端电压的上下限：
该电路只能输出0-5V信号



PIN CONNECTIONS



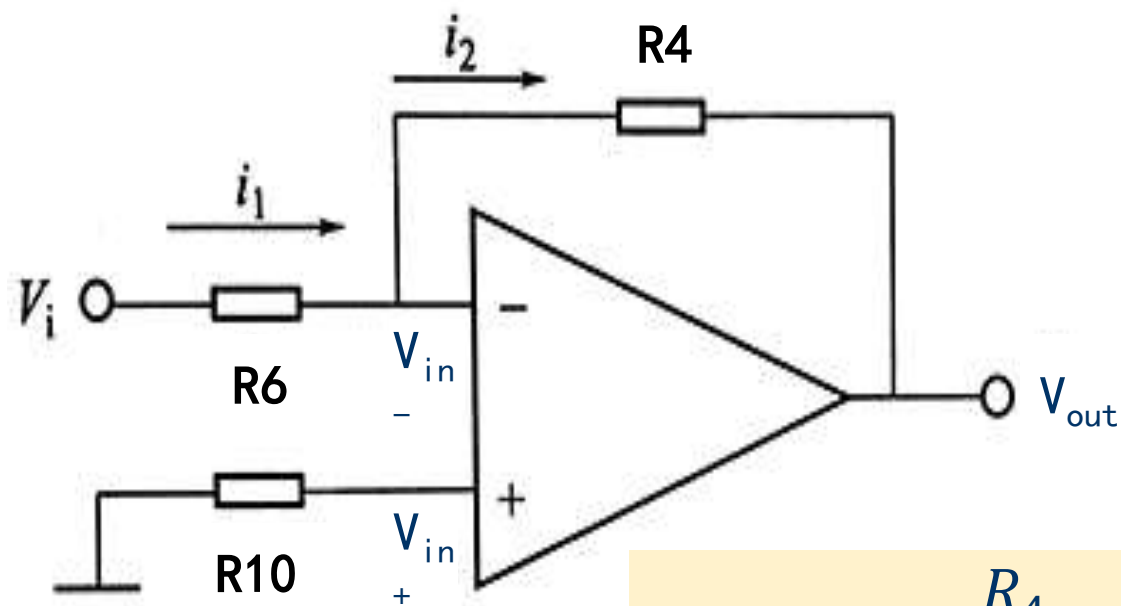
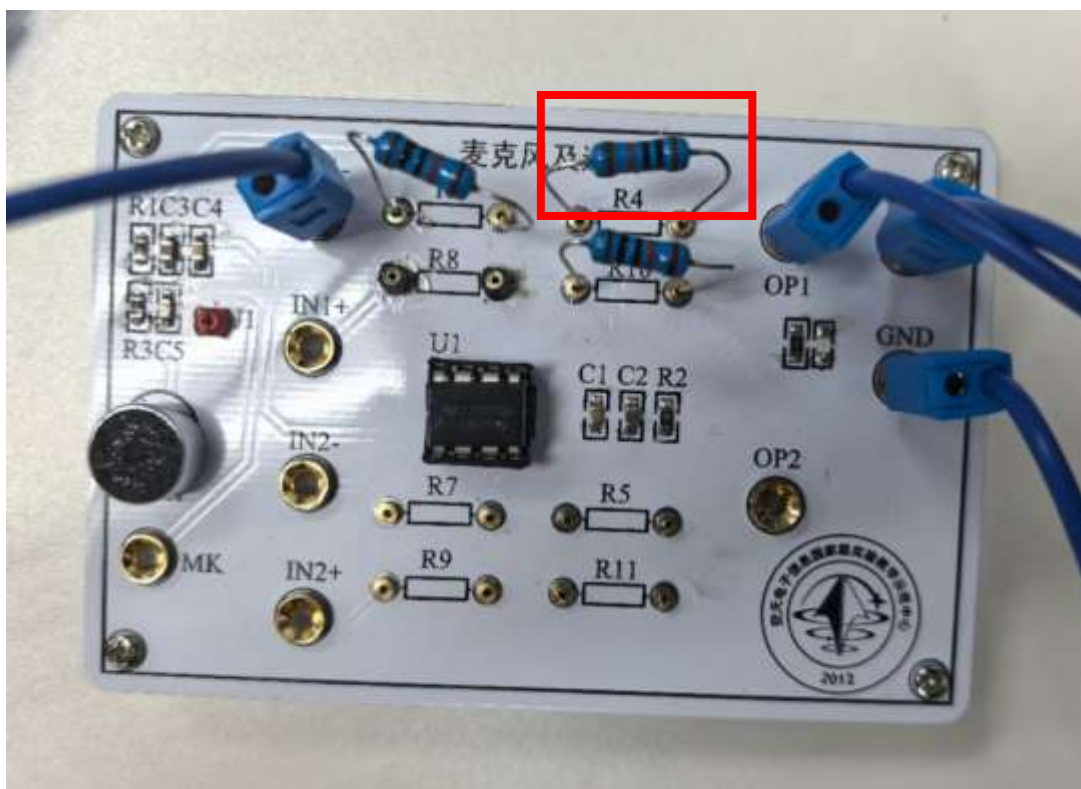


实验二：测试单级放大电路输出

■ 搭建反相放大电路，观察对正弦波的放大效果

➤ 接入电阻 $R_6 = R_{10} = 10\text{k}\Omega$

➤ 接入反馈电阻 $R_4 = 82\text{k}\Omega$ ，计算易得输出信号约为输入信号的8倍



$$V_{out} = -\frac{R_4}{R_6} V_i$$



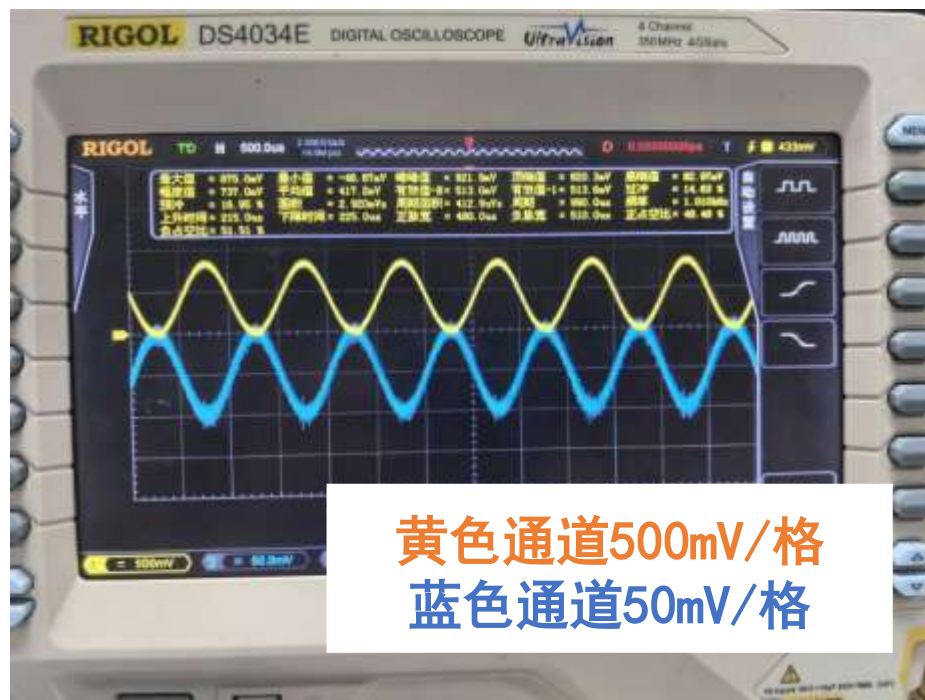
实验二：测试单级放大电路输出

■ 搭建反相放大电路，观察对正弦波的放大效果

- 信号源输出偏移量设置为 -50mV ，幅度 100mVpp ，观察并计算电路输出相较于输入的放大倍数



信号源输出偏移量设置为 -50mV
幅度 100mVpp



放大后的正弦波（黄色）



实验三：控制信号的直流分量

- 改变信号发生器的直流偏移，观察示波器的波形
 - 将信号发生器的输出偏移量 V_{DC} 依次设为
0mV, 50mV, -50mV, -100mV, -200mV, -500mV

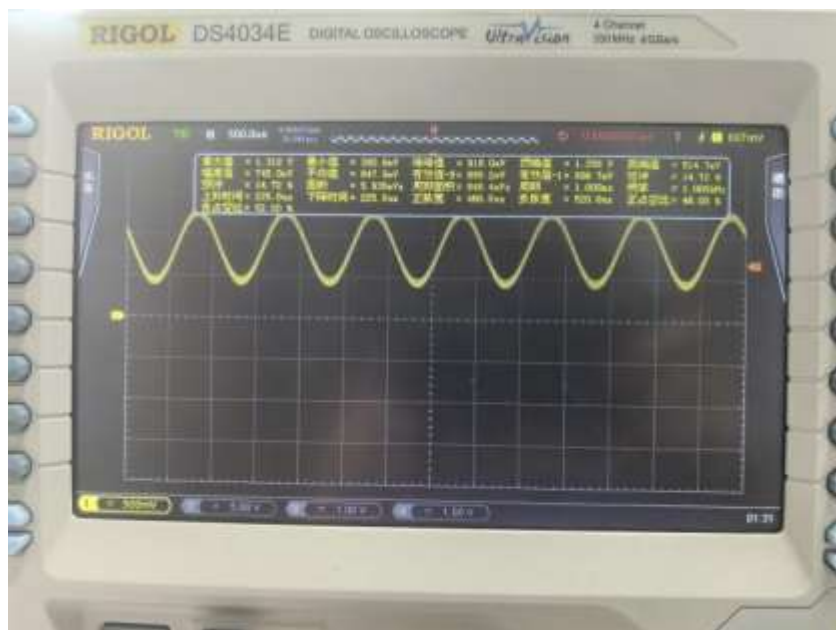




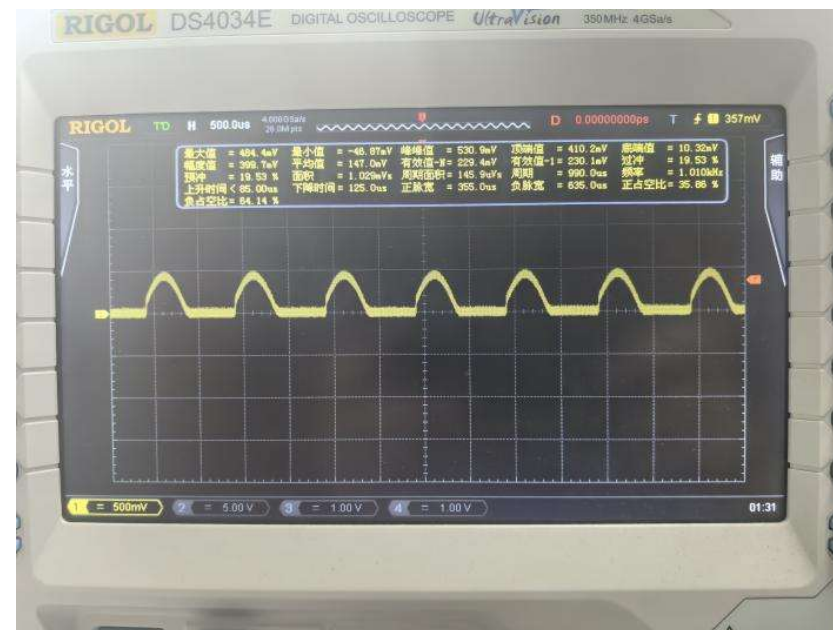
实验三：控制信号的直流分量

■ 改变信号发生器的直流偏移，观察示波器的波形

- $V_{DC}=0\text{mV}$ 或 50mV 时，波形底部被削平，无法降低到 0V 以下
- $V_{DC}=-200\text{mV}$ 或 -100mV 时，波形保持完整，且 V_{DC} 越向负方向移动，输出波形的平均值（直流分量）越向正方向移动



直流偏置-100mV



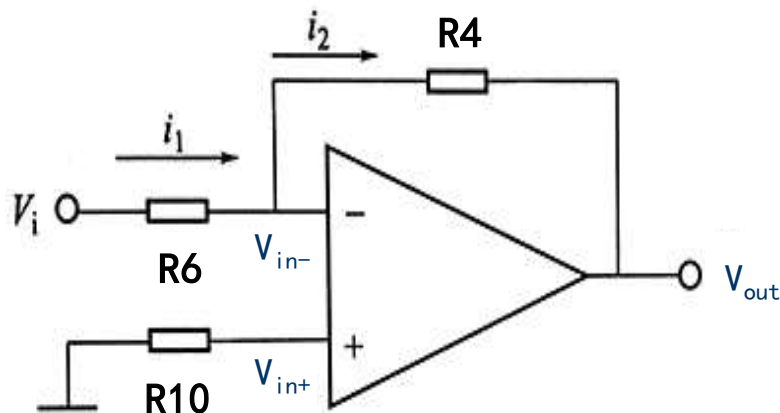
直流偏置0mV



实验三：控制信号的直流分量

■ 现象分析：

- 当 $V_{DC}=0\text{mV}$ 时， V_i 在 $-50\text{mV}\sim 50\text{mV}$
 - 反向放大后 ($R_4/R_{10}=8.2$) V_{out} 理论值为 $-410\text{mV}\sim +410\text{mV}$
- 当 $V_{DC}=50\text{mV}$ 时， V_i 在 $0\text{mV}\sim 100\text{mV}$ 之间
 - 反向放大后 V_{out} 理论值为 $-820\text{mV}\sim 0\text{mV}$ 之间
- 两种情况都有小于 0V 的部分，小于 0V 的部分固定输出 0V
 - 电源电压决定输出端电压的上下限，该电路只能输出 $0\text{--}5\text{V}$ 信号



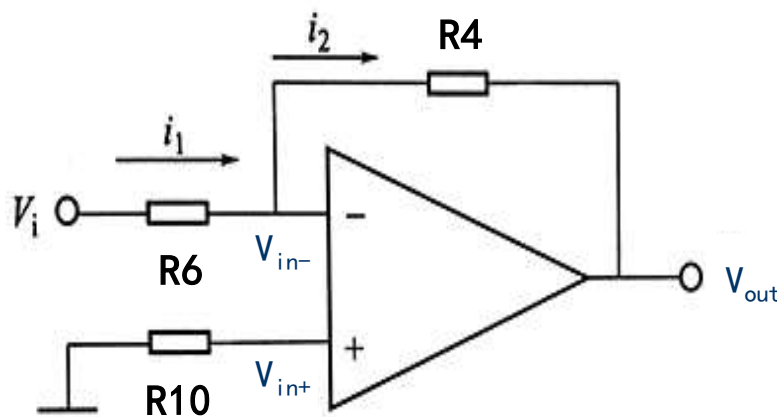
$$V_{out} = -\frac{R_4}{R_6} V_i$$



实验三：控制信号的直流分量

■ 现象分析：

- 当 $V_{DC} = -200\text{mV}$ 时， V_i 在 $-250\text{mV} \sim -150\text{mV}$
 - 反向放大后 ($R_4/R_{10}=8.2$) V_{out} 理论值为 $1.23\text{V} \sim +2.05\text{V}$
- 当 $V_{DC} = -100\text{mV}$ 时， V_i 在 $-150\text{mV} \sim -50\text{mV}$ 之间
 - 反向放大后 V_{out} 理论值为 $410\text{mV} \sim 1.23\text{V}$ 之间
- 两种情况都没有小于 0V 的部分，均可正常放大



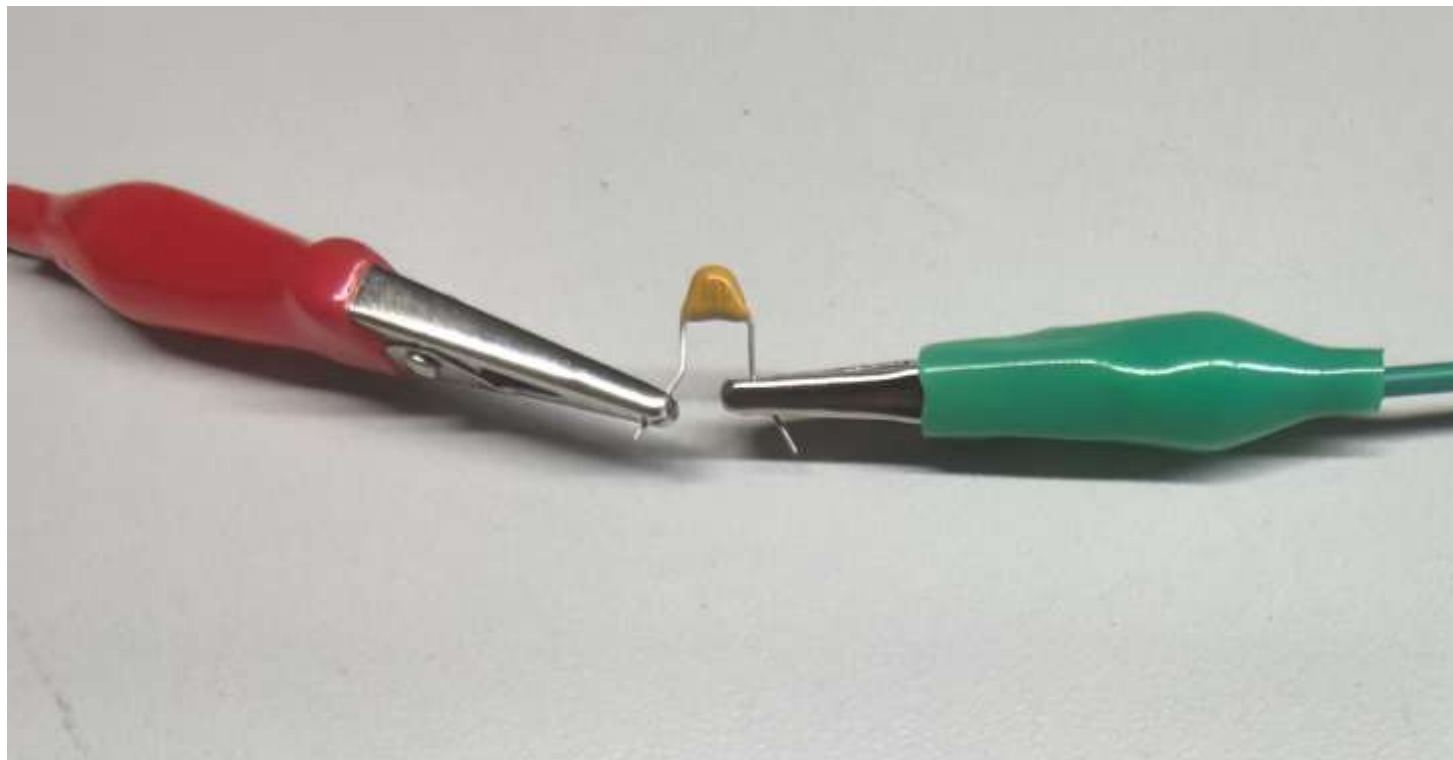
$$V_{out} = -\frac{R_4}{R_6} V_i$$



实验三：控制信号的直流分量

■ 滤除输入信号的直流分量

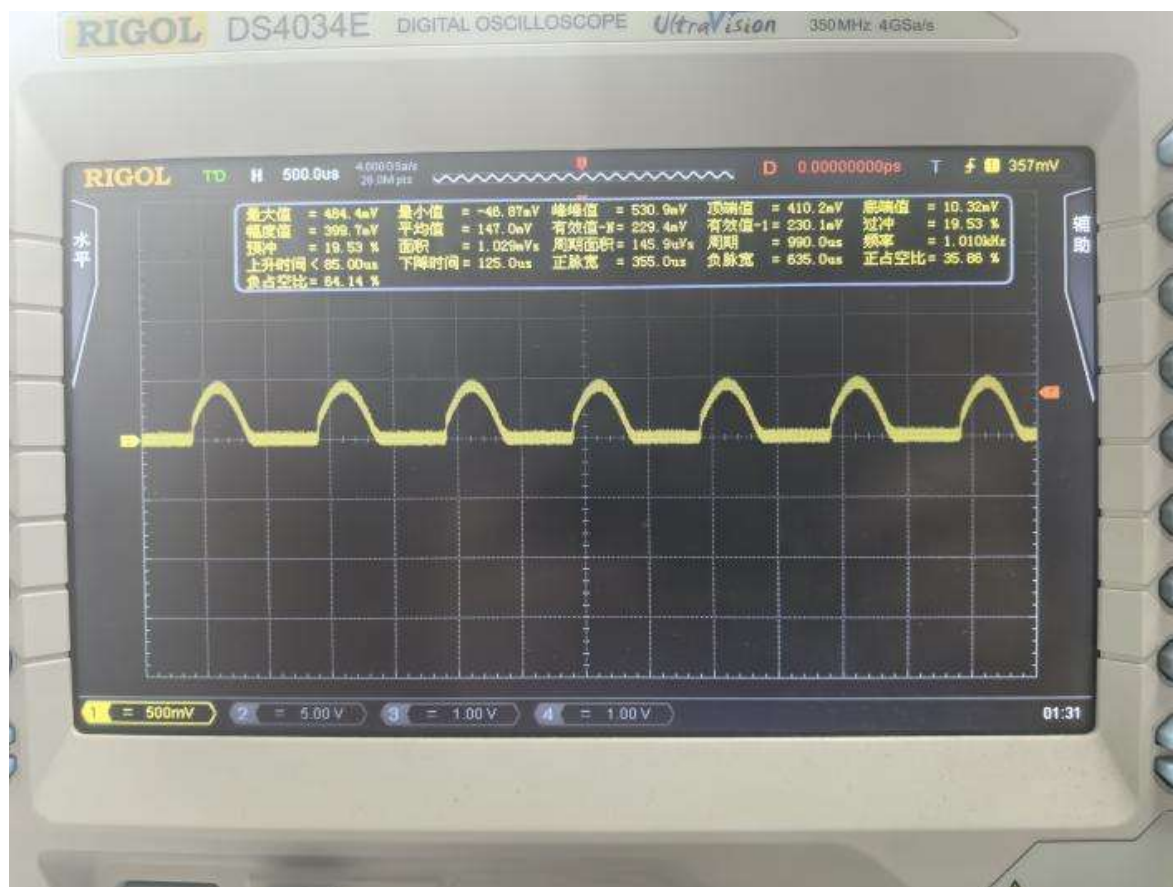
- 在信号发生器输出端与运放反相输入端IN1-之间接一个电容
- 目的：排除输入信号中直流分量对输出的影响





实验三：控制信号的直流分量

- 再次改变信号发生器的直流偏移，观察示波器的波形
 - 由于电容滤除了直流分量，现在输出信号不再随 V_{DC} 变化

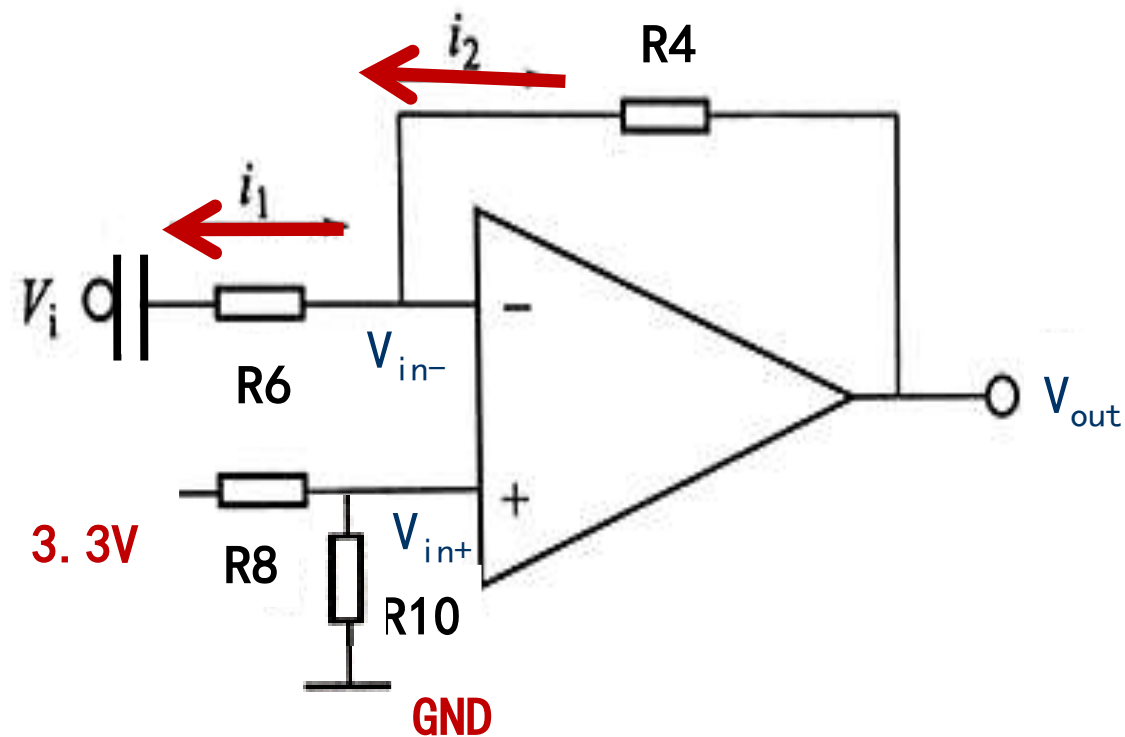




实验三：控制信号的直流分量

■ 将运放同向端输入设置为直流电压

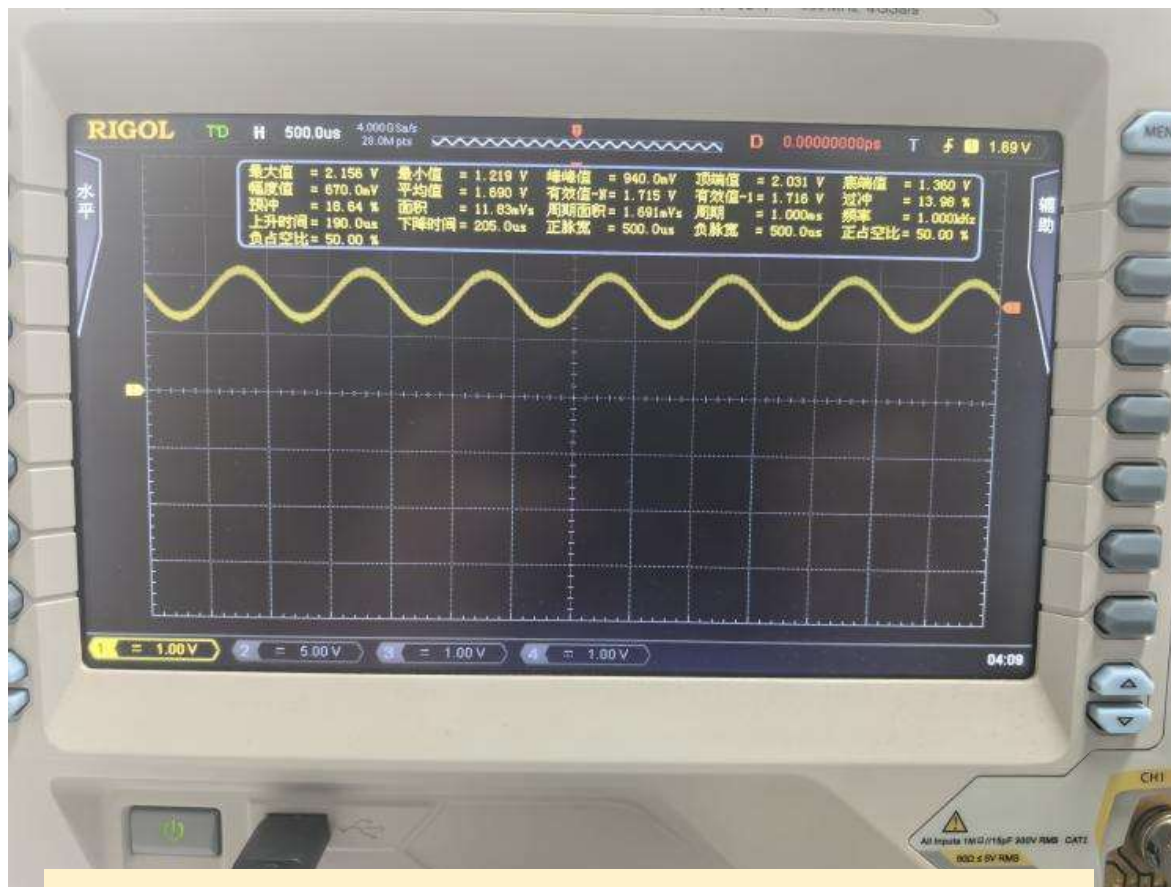
- IN1+端口接入3.3V电压，接入电阻 $R8=10k\ \Omega$
- 目的：通过改变 V_{in+} 端口的电压，使输出电压全部在0~5V范围内



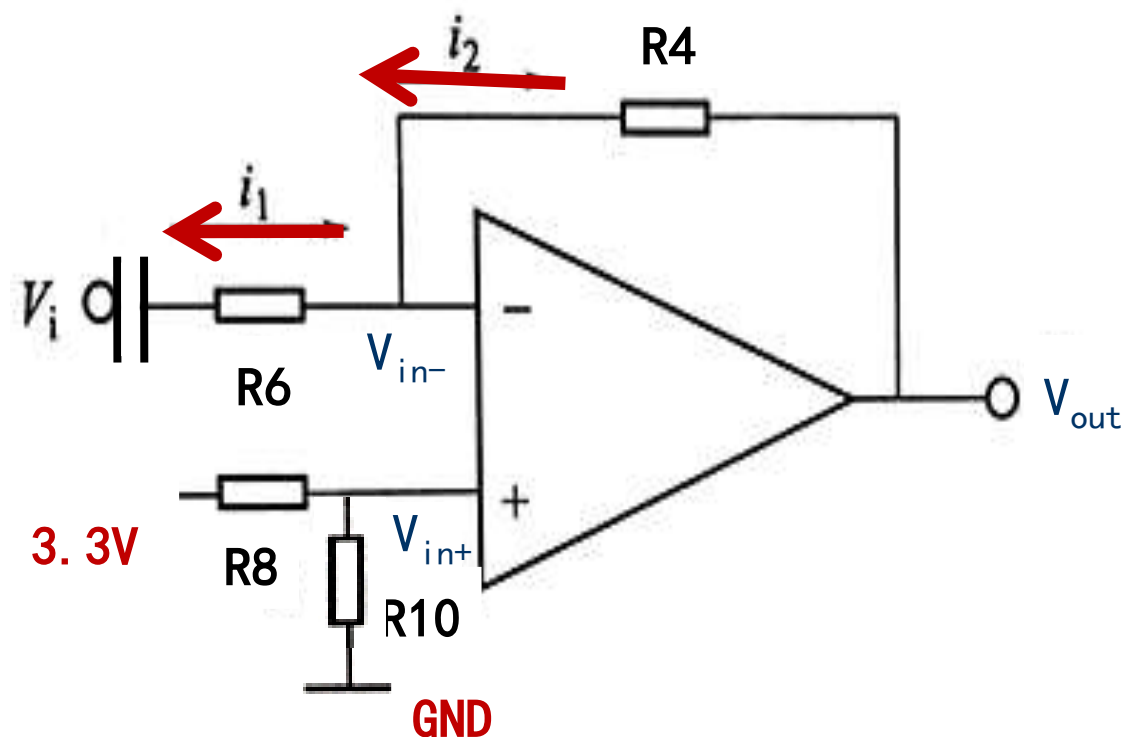


实验三：控制信号的直流分量

■ 输出信号被抬高，底部削波现象消失



$$V_{out} = -\frac{R_4}{R_6} V_i + \frac{R_{10}}{R_{10} + R_8} * 3.3$$

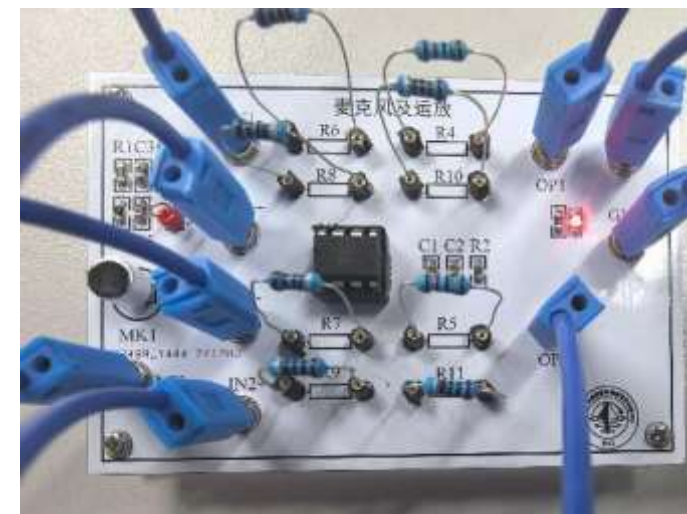




实验四：Arduino采集语音信号

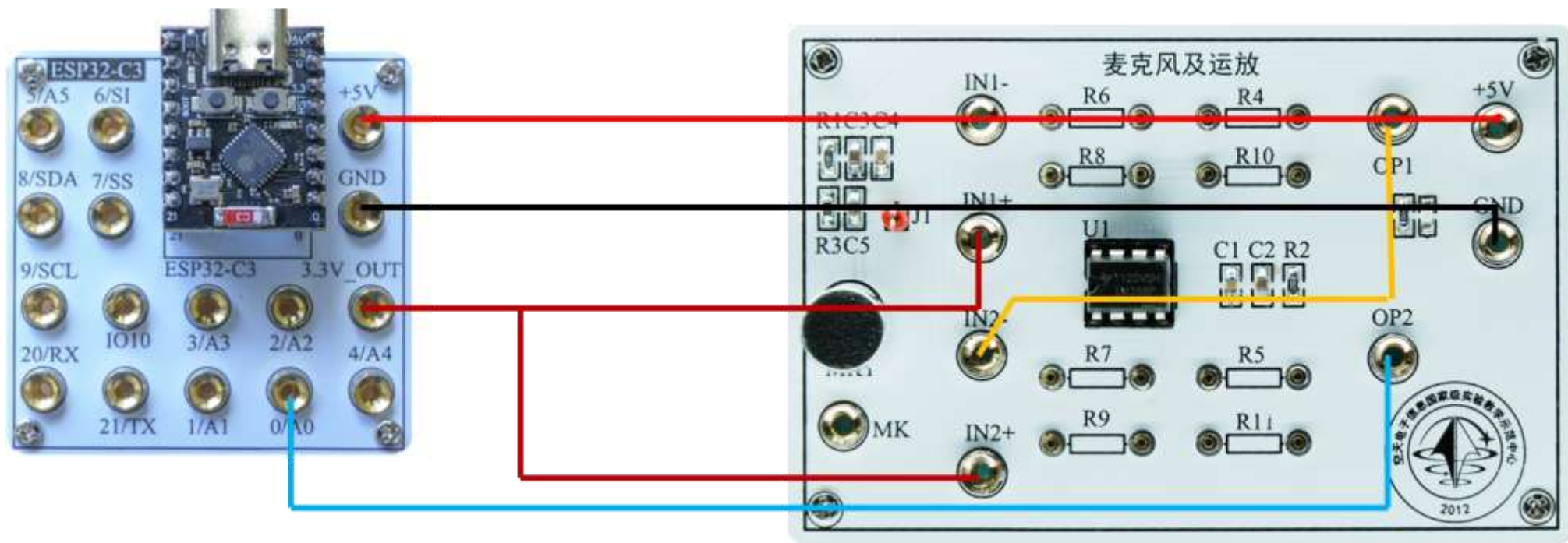
■ 实验电路

- 搭建第二级运算放大电路，R5 ($82\text{k}\Omega$)，R7 ($10\text{k}\Omega$)，R9 ($10\text{k}\Omega$)，R11 ($10\text{k}\Omega$)
- 将ESP32的3.3V_OUT引脚接到第二级同相输入端IN2+
- 传声器输出MK1连接到第一级放大器反相输入端IN1-
- 第一级输出OP1连接到第二级放大器反相输入端IN2-
- OP2输出放大倍率为67倍，输出信号直流分量为1.65V的语音信号
- 使用示波器观察MK1和OP2输出的语音信号
- 将OP2连接到ESP32板的A0端口





实验四：Arduino采集语音信号





实验四：Arduino采集语音信号

实验代码

```
1  #include <Arduino.h>
2
3  // 定义引脚
4  const int AUDIO_IN = 0; // 选择 GPIO 0 作为模拟输入
5
6  //-----
7  // 全局常量
8  //-----
9
10 const byte sample_interval = 125; //采样时间间隔125us, 即采样率8kHz
11 const byte scale = 4; //幅度缩放因子
12 const byte thresh = 100; //音量门限
13 const int sample_length = 8000; //采集8000个样本, 1秒时间长度
14
15 //-----
16 // 全局变量
17 //-----
18
19 int adcValue = 0;
20 long curData[sample_length];
21
22 //-----
23 // ADCsimple: 简单读取 ADC 的示例
24 //-----
25 void ADCsimple(void) {
26     adcValue = analogRead(AUDIO_IN); // 从 AUDIO_IN 读取模拟值
27     Serial.println(adcValue); // 打印 ADC 结果
28 }
29
30 //-----
31 // ADCfast: 以8kHz采样率读取 ADC 的值
32 //-----
33 bool ADCfast(void) {
34     static int zero = 580;
35     static int curSample = 7999;
36     static unsigned long prevTime;
37     long val = 0;
38
39     bool Collecting = false;
40
41     for (curSample = 0; curSample < sample_length; curSample++) {
42         prevTime = micros();
43         adcValue = analogRead(AUDIO_IN); // 读取模拟信号
44
45         val = adcValue/scale; // 将ADC采样值进行尺度缩放
46
47         //去除直流偏置值
48         if (val < zero)
49             zero--;
50         else
51             zero++;
52
53         val = val - zero;
54
55         //判断音量是否超过阈值, 超过开始采集语音信号
56         if (!Collecting) {
57             if (abs(val) > thresh)
58                 Collecting = true;
59             else
60                 return false;
61         }
62
63         curData[curSample] = val;
64         while (micros() - prevTime < sample_interval); //等待下一次采样
65     }
66     //采样完毕通过串口回传数据
67     if (Collecting) {
68         for (curSample = 0; curSample < sample_length; curSample++) {
69             Serial.println(curData[curSample]);
70             Serial.write(curData[curSample]);
71             Serial.write((curData[curSample]>>8));
72         }
73     }
74     return Collecting;
75 }
76
77 //-----
78 // 初始化
79 //-----
80 void setup(void) {
81     Serial.begin(115200); // 初始化串口通信
82
83     pinMode(AUDIO_IN, INPUT); // 设置 GPIO 3 为输入
84 }
85
86 //-----
87 // 主循环
88 //-----
89 void loop(void) {
90     ADCsimple(); // 简单模式测试
91     ADCfast(); // 快速模式测试
92     delay(10); // 增加延时, 避免输出过快
93 }
94 }
```



实验四：Arduino采集语音信号

■ 代码解释

```
//-----  
// 全局常量  
//-----  
  
const byte sample_interval = 125; //采样时间间隔125us，即采样率8kHz  
const byte scale = 4; //幅度缩放因子  
const byte thresh = 100; //音量门限  
const int sample_length = 8000; //采集8000个样本，1秒时间长度
```

人类声音可以达到的最高频率约为4kHz，根据奈奎斯特采样定理，把采样率设定为8kHz。

```
if (!Collecting) {  
    if (abs(val) > thresh)  
        Collecting = true;  
    else  
        return false;  
}
```

判断音量是否超过阈值，超过开始采集语音
可提高thresh值以防噪声干扰



实验四：Arduino采集语音信号

■ 代码解释

```
for (curSample = 0; curSample < sample_length; curSample++) {  
    prevTime = micros();  
    adcValue = analogRead(AUDIO_IN); // 读取模拟信号  
  
    val = adcValue/scale; // 将ADC采样值进行尺度缩放  
  
    //去除直流偏置值  
    if (val < zero)  
        zero--;  
    else  
        zero++;  
  
    val = val - zero;  
  
    //判断音量是否超过阈值：超过开始采集语音信号  
    if (!Collecting) {  
        if (abs(val) > thresh)  
            Collecting = true;  
        else  
            return false;  
    }  
  
    curData[curSample] = val;  
    while (micros() - prevTime < sample_interval); //等待下一次采样  
}
```

使用内置的micros()函数作为定时器，单位为微秒。程序规定相邻两次采样间隔125微秒，等同于8kHz的采样率。



实验四：Arduino采集语音信号

■ 代码解释

```
54
55 //判断音量是否超过阈值：超过开始采集语音信号
56 if (!Collecting) {
57     if (abs(val) > thresh)
58         Collecting = true;
59     else
60         return false;
61 }
62
63 curData[curSample] = val;
64 while (micros() - prevTime < sample_interval); //等待下一次采样
65 }
66 //采样完毕通过串口回传数据
67 if (Collecting) {
68     for (curSample = 0; curSample < sample_length; curSample++) {
69         Serial.println(curData[curSample]);
70         Serial.write(curData[curSample]);
71         Serial.write((curData[curSample]>>8));
72     }
73 }
74 return Collecting;
75 }
76
```

使用Serial.write()函数将每个采样点数据分2个字节上传至上位机软件。



实验四：Arduino采集语音信号

■ 开启USB CDC后，将实验代码验证并上传至开发板

The screenshot shows the Arduino IDE 2.3.3 interface. The 'Tools' menu is open, and 'ESP32C3 Dev Module' is selected. The code editor displays the 'speechCollection.ino' file with the following code:

```
40
41 for (curSample = 0; curSample < sample_length; curSample++) {
42   prevTime = micros();
43   adcValue = analogRead(AUDIO_IN); // 读取模拟信号
44
45   val = adcValue/scale; // 将ADC采样值进行尺度缩放
46
47   // 去除直流偏置值
48   if (val < zero)
49     zero--;
50   else
51     zero++;
52
53   val = val - zero;
54
55   //判断音量是否超过阈值: 超过开始采集语音信号
56   if (!Collecting) {
57     if (abs(val) > thresh)
58       Collecting = true;
59     else
60       return false;
61   }
62
63   curData[curSample] = val;
64   while (micros() - prevTime < sample_interval); //等待下一次采样
65 }
66 //采样完毕通过串口回传数据
67 if (Collecting) {
68   for (curSample = 0; curSample < sample_length; curSample++) {
69     Serial.println(curData[curSample]);
70     // Serial.write(curData[curSample]);
71     // Serial.write((curData[curSample]>>8));
72   }
73 }
74 return Collecting;
75 }
76
77 //-----
78 // 30/10/2023
```

The screenshot shows the 'Tools' menu in the Arduino IDE 2.3.3. The 'USB CDC On Boot: Enabled' and 'Flash Mode: DIO' options are highlighted with red boxes. The menu items are as follows:

- 自动格式化 (Ctrl+T)
- 项目存档
- 管理库... (Ctrl+Shift+I)
- 串口监视器 (Ctrl+Shift+M)
- 串口绘图仪
- 固件更新
- 上传 SSL 根证书
- 开发板: "ESP32C3 Dev Module"
- 端口: "COM5"
- 获得开发板信息
- USB CDC On Boot: "Enabled"
- CPU Frequency: "160MHz (WiFi)"
- Core Debug Level: "None"
- Erase All Flash Before Sketch Upload: "Enabled"
- Flash Frequency: "80MHz"
- Flash Mode: "DIO"
- Flash Size: "4MB (32Mb)"
- JTAG Adapter: "Disabled"
- Partition Scheme: "Default 4MB with spiffs (1.2MB APP/1.5MB SPIFFS)"
- Upload Speed: "921600"
- 编程器
- 烧录引导程序



实验四：Arduino采集语音信号

■ 打开上位机SpeechLearn，连接好串口后采集音频

- 设置波特率为115200，选择好串口后点击Connect连接，出现“串口已连接”的提示后点击“开始采集”



上位机主界面

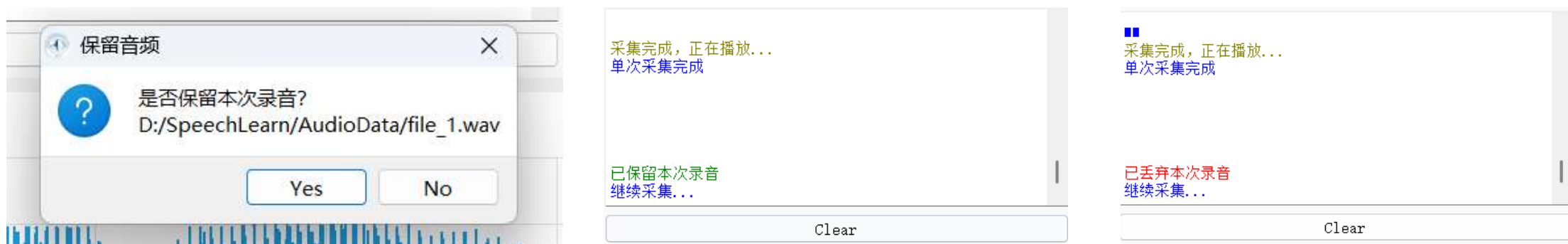


连接开发板

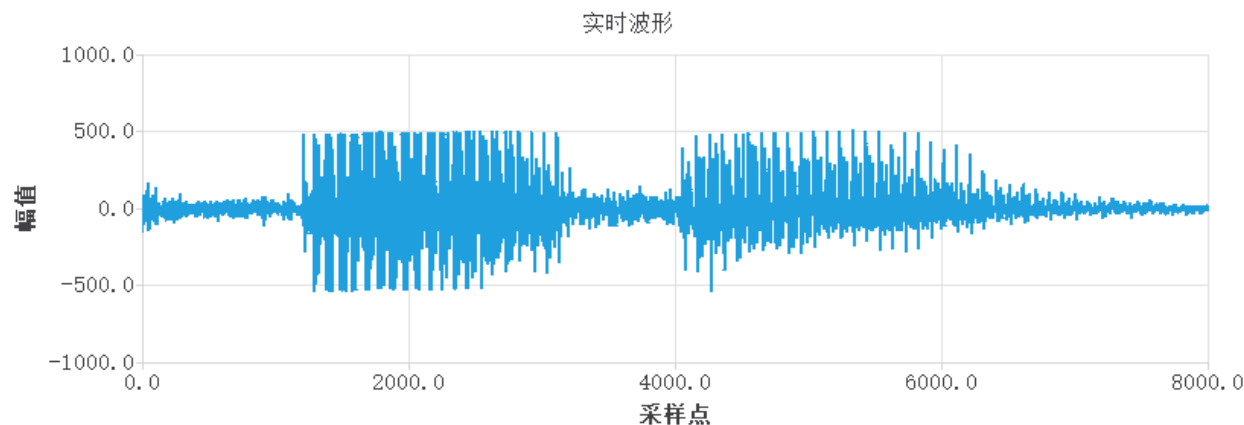


实验四：Arduino采集语音信号

- 单次采集结束后上位机会回放所采集的语音信号，据此可保留此次采集数据，也可丢弃此次数据后重新采集



- 在界面下方的图表中也可以直观地看到采集的语音信号





实验四：Arduino采集语音信号

- 一系列采集结束后点击“停止采集”结束本次连续采集，在上位机程序所在目录下会生成一个文件夹AudioData，其中存有一次连续采集中若干次单次采集得到的语音文件

AudioData

2026/3/12 1:07

文件夹

file_1.wav
file_2.wav
file_3.wav
file_4.wav
file_5.wav
file_6.wav
file_7.wav
file_8.wav
file_9.wav
file_10.wav

采集完成，正在播放...
单次采集完成

已丢弃本次录音
继续采集...
连续采集已停止

Clear



实验四：Arduino采集语音信号

- 对传声器说出以下短语，截图记录采集结果，并播放保证语音采集正确

➤ 开灯

Guandeng

2026/3/12 1:07

文件夹

➤ 关灯

Kaideng

2026/3/12 1:07

文件夹

➤ 你好

Nihao

2026/3/12 1:07

文件夹

➤ 再见

Zaijian

2026/3/12 1:07

文件夹

- 每个短语至少采集20个对应的语音文件。每次采集完一个短语，将AudioData文件夹重命名为该短语的拼音（如Kaideng）后再进行下一个短语的采集，直到四个短语全部采集完毕。保存好这四组语音文件，会作为后续实验的数据